République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique Université de Sousse

Institut Supérieur des Sciences Appliquées et de Technologie de Sousse

Departement Informatique

RAPPORT DE MINI PROJET WEB

FIA3

Tchanchinet

Élaboré par :

Nermine Galai
Sarra Gharbi

A.U. : 2026/2027

Plan

1. Introduction **3**

2. Cahier des charges / Objectifs **3**

3. Conception **3**

4. Implémentation (Code et explications) **5**

5. Résultat final **9**

6. Difficultés rencontrées et solutions **16**

7. Conclusion **16**

Références **16**

1. Introduction

Dans le cadre de notre mini-projet de développement web, nous avons décidé de developper un quiz inspiré des devinettes tunisiennes présentées dans l'ouvrage *Énigmes de Tunisie* de Tahar Fazaa [1]. Ce projet a pour objectif de mettre en pratique et de tester nos compétences en HTML, CSS et JavaScript.

2. Cahier des charges / Objectifs

L'objectif de ce projet est de développer un site web interactif qui permet aux utilisateurs de participer à un quiz et de suivre leurs résultats en temps réel. L'interface fournit un retour visuel après chaque réponse et affiche le score final à la fin du quiz. Le site devra être développé en HTML5, CSS3 et JavaScript, proposer une page d'accueil ainsi qu'une interface de quiz, tous les pages doivent être responsive. Il devra également prendre en compte l'accessibilité, intégrer des animations CSS tout en respectant les standards du W3C.

3. Conception

Structure du dossier :

```
Tchanchinet/
├── assets/
│   └── tunisian_riddles_40.json
├── css/
│   ├── styles.css
│   ├── index.css
│   ├── quiz-shared.css
│   ├── quiz.css
│   ├── results.css
│   ├── versus.css
│   ├── versus-quiz.css
│   └── versus-results.css
├── js/
│   ├── results.js
│   ├── quiz.js
│   ├── versus-quiz.js
│   └── versus-results.js
├── pages/
│   ├── quiz.html
│   ├── results.html
│   ├── versus.html
│   ├── versus-quiz.html
│   └── versus-results.html
├── index.html
├── README.md
└── .gitignore
```

Le projet est organisé en plusieurs dossiers afin de séparer les différents composants du site et de faciliter sa maintenance.

- **assets/** : contient les ressources utilisées dans le site, notamment les différentes images et icônes, ainsi que les devinettes, leurs indices et leurs réponses.
- **css/** : regroupe tous les fichiers CSS.
- **js/** : regroupe les scripts JavaScript.
- **pages/** : contient les pages HTML du quiz et des résultats (mode solo et versus).
- **index.html** : représente la page d'accueil du site.

Parcours de navigation

Le site propose deux parcours, depuis la page d'accueil :

- **Mode individuel :** `index.html` → `quiz.html?mode=solo` → `results.html`.
- **Mode défi :** `index.html` → `versus.html` → `versus-quiz.html` → `versus-results.html`.

Structure HTML

Les pages du site sont développées en HTML5 et suivent une structure organisée :

- `header` : contient le logo du site sur toutes les pages. Dans les interfaces de quiz, il affiche également les informations du jeu, comme le score, le temps restant ou le nombre de tentatives.
- `nav` : permet de naviguer entre les différentes pages du site, notamment de revenir à la page d'accueil, ainsi que de choisir entre les différents modes de jeu.
- `main` : regroupe le contenu principal de chaque page.
- `section` : permet d'organiser le contenu en différentes parties, comme les questions, les indices et les scores des joueurs.
- `form` : permet la saisie des noms des joueurs dans le mode versus.

Nous n'avons pas utilisé de `footer`, car nous n'en avions pas besoin dans notre site.

Charte graphique

Pour le design du site, nous avons choisi de nous inspirer du style traditionnel tunisien afin de créer une ambiance qui rappelle notre culture et notre patrimoine.

- `Thème visuel` : nous avons utilisé une image de portes tunisiennes comme arrière-plan du site. Nous avons également ajouté des éléments traditionnels comme la chéchia, la khomsa et le jasmin.
- `Couleurs` : nous avons principalement choisi le bleu et le blanc, qui créent une ambiance traditionnelle et chaleureuse.
- `Polices` : nous avons utilisé des polices simples et lisibles adaptées à l'affichage du texte en arabe.

Outils utilisés

- `Visual Studio Code` : pour écrire et organiser le code du projet.
- `Microsoft Edge` : pour tester et visualiser les différentes pages du site.
- `Live Server` : pour lancer le site en local afin d'effectuer les tests avec Lighthouse.

• `Google Gemini` : pour générer certaines images utilisées dans le design du site.

4. Implémentation (Code et explications)

4.1 Structure de la page et interface en arabe

Les pages du jeu déclarent l'arabe comme langue et utilisent une direction droite-à-gauche. La structure HTML s'appuie sur des éléments sémantiques et intègre des aides à l'accessibilité, comme le lien d'évitement et le label associé au champ de réponse.

```
<html lang="ar" dir="rtl">
<body class="quiz-page">
  <a class="skip-link" href="#main">...</a>
  <div id="quiz" class="game-screen"
       data-attempts-left="3"
       data-status="ready"
       data-hint-state="hidden">
    <header class="game-header">...</header>
    <main id="main" class="quiz-main">
      <section class="ornament-panel question-panel">
        <h1 id="question-text">...</h1>
      </section>
      <label class="visually-hidden" for="answer">...</label>
      <input id="answer" type="text" dir="rtl"
             autocomplete="off">
      <button id="check-answer" type="button">...</button>
    </main>
  </div>
</body>
```

Les attributs `lang="ar"` et `dir="rtl"` définissent la langue et le sens de lecture. Le lien d'évitement permet d'accéder directement au contenu principal. Le label, masqué visuellement, reste accessible aux lecteurs d'écran.

4.2 Formulaire de saisie des noms (mode deux joueurs)

Le formulaire de préparation du défi valide les deux noms en HTML et les transmet à la page du quiz avec la méthode GET.

```
<form id="players-form" action="versus-quiz.html"
      method="get">
  <input type="hidden" name="mode" value="versus">

  <label for="player-one-name">...</label>
  <input id="player-one-name" name="player1"
         type="text" maxlength="24"
         pattern=".*\S.*" required>
```

```
  <label for="player-two-name">...</label>
  <input id="player-two-name" name="player2"
       type="text" maxlength="24"
       pattern=".*\S.*" required>

  <button type="submit">...</button>
</form>
```

L'attribut `required` rend les deux champs obligatoires, `pattern` efuse un nom composé uniquement d'espaces et `maxlength` limite sa longueur à 24 caractères. Le script du quiz relit les paramètres de l'URL, puis vérifie et limite à nouveau les noms

4.3 Gestion des états avec les attributs data-* Le JavaScript met à jour les attributs de #quiz ; le CSS utilise ces états pour afficher les retours visuels, les tentatives restantes et l'indice.

```
[data-attempts-left="2"] [data-attempt="1"]
.chechia-red {
  display: none;
}

[data-attempts-left="2"] [data-attempt="1"]
.chechia-gray {
  display: block;
}

[data-status="wrong"] .feedback-wrong {
  display: block;
}

[data-status="correct"] .feedback-correct {
  display: block;
}

[data-status="exhausted"] .feedback-exhausted {
  display: block;
}

[data-attempts-left="1"]
[data-hint-state="revealed"] .hint-content {
  display: block;
}
```

Le JavaScript modifie ces attributs selon les actions du joueur :

```
quiz.dataset.attemptsLeft = String(attemptsLeft);
quiz.dataset.status = "wrong";
quiz.dataset.hintState = "revealed";
```

La séparation entre logique et présentation facilite la maintenance : le script modifie l'état du jeu, tandis que les règles CSS choisissent les éléments visibles. Après deux erreurs, il reste une tentative et l'indice peut être débloqué ; il n'apparaît qu'après activation de la khomsa.

4.4 Couleurs des joueurs et responsive

Les deux joueurs partagent les mêmes règles de présentation. Des variables CSS définissent leurs couleurs respectives ; des unités adaptatives et des media queries permettent d'ajuster l'interface aux différentes tailles d'écran.

```
.player-blue {
  --player-color: #3e79aa;
  --player-text: #244b7a;
  --player-background: #e4f0fa;
}

.player-gold {
  --player-color: #bf962f;
  --player-text: #795516;
  --player-background: #faf0d6;
}

.player-card {
  border: 1px solid var(--player-color);
  border-radius: 12px;
  background: var(--player-background);
  color: var(--player-text);
}
```

La classe `player-card` est commune aux deux joueurs. Seules les variables de couleur changent.

4.5 Adaptation aux différentes tailles d'écran

Pour rendre le site responsive, nous utilisons des media queries et des tailles adaptatives. Cela permet d'ajuster la disposition et la taille des éléments selon les dimensions de l'écran.

```
.game-screen {
  width: 100%;
  max-width: 1380px;
}

@media (max-width: 1100px) {
  .game-screen {
    max-width: 1000px;
    padding: 8px 12px;
  }
}
```

Les media queries permettent de modifier la disposition des éléments sur les écrans plus petits, tandis que les propriétés `width` et `max-width` permettent d'adapter leur largeur.

4.6 Logique du jeu en JavaScript

Les questions du jeu sont chargées depuis un fichier JSON contenant 40 devinettes tunisiennes. Chaque devinette possède une question, un indice et une liste de réponses acceptées.
Avant de comparer la réponse du joueur avec celles du fichier JSON, nous utilisons une fonction pour supprimer les espaces inutiles.

```
function normalizeAnswer(value) {
  return value.trim().replace(/\s+/g, " ");
}
```

Cette fonction supprime les espaces au début et à la fin du texte, ainsi que les espaces répétés entre les mots.
Chaque question donne au joueur trois tentatives. Le nombre de points dépend de la tentative utilisée.

```
const POINTS_BY_ATTEMPT = [100, 80, 50];
const POINTS_WITH_HINT = 25;
const MAX_ATTEMPTS = 3;
```

Une bonne réponse rapporte 100 points à la première tentative, 80 à la deuxième et 50 à la troisième. Si le joueur utilise l'indice, il obtient seulement 25 points en cas de bonne réponse.
Après trois mauvaises réponses, la réponse correcte est affichée. Le joueur peut également passer une question sans gagner de points.
Dans le mode solo, le joueur dispose de trois minutes pour répondre au maximum de questions possible. Les questions sont affichées dans un ordre aléatoire.

```
const TOTAL_TIME = 3 * 60;

timerInterval = setInterval(() => {
  timer--;
  timerDisplay.textContent = formatTime(timer);

  if (timer <= 0) {
    finishGame("time");
  }
}, 1000);
```

Le temps restant est mis à jour chaque seconde. La partie se termine lorsque le temps est écoulé.
Les joueurs alternent après chaque question terminée ou passée. Chacun répond à cinq questions ; une bonne réponse rapporte 100, 80 ou 50 points selon la tentative, et seulement 25 points si l'indice a été utilisé.

```
setActivePlayer(turnsPlayed % 2 === 0 ? 1 : 2);

sessionStorage.setItem("versusResult", JSON.stringify({
  player1: { name: players[1].name, score: players[1].score },
  player2: { name: players[2].name, score: players[2].score },
}));
window.location.href = "versus-results.html";
```

Les scores et les noms sont conservés dans `sessionStorage` jusqu'à l'affichage des résultats. La page de résultats compare les deux scores et met à jour l'attribut `data-winner` avec `1`, `2` ou `draw`.

5. Résultat final

Le site propose un parcours complet autour des devinettes tunisiennes. La page d'accueil présente les règles et donne accès au jeu individuel ou au défi à deux. En mode individuel, le joueur répond aux questions avant la fin du compte à rebours ; en mode défi, les deux joueurs jouent à tour de rôle sur le même appareil. Les captures suivantes illustrent les principales étapes du jeu.
L'interface en arabe, orientée de droite à gauche, reprend des motifs traditionnels dans une palette dominée par le bleu et le blanc. Les indices, les tentatives et les scores disposent de retours visuels, tandis que les couleurs bleu et or distinguent les joueurs du mode défi.



Page d'accueil : règles et choix du mode de jeu.



Début du quiz individuel : énigme, temps et tentatives disponibles.



Retour visuel après une première réponse incorrecte.



Après deux erreurs, l'indice est débloqué et peut être consulté.



Affichage de l'indice sous la question.



Réponse correcte et mise à jour du score après utilisation de l'indice.



Poursuite de la partie individuelle avec le score actualisé.



Écran final du mode solo et total des points obtenus.



Formulaire de saisie des noms avant le défi à deux joueurs.



Défi : affichage des joueurs et de la question en cours.



Tour suivant du défi avec les scores des deux joueurs.



Résultat du défi : scores finaux et gagnant.

Dépôt GitHub : `https://github.com/Nermine-Galai/Tchanchinet`

6. Difficultés rencontrées et solutions

- **Comparaison des réponses en arabe :** une même réponse pouvait être écrite de plusieurs façons. Pour résoudre ce problème, nous avons ajouté une liste de réponses acceptées pour chaque devinette dans le fichier JSON. Une fonction de normalisation permet également de supprimer les espaces inutiles avant la comparaison.
- **Gestion des états du quiz :** il fallait réinitialiser les tentatives, l'indice et les commandes à chaque nouvelle énigme, tout en empêchant une réponse ou un score d'être enregistré plusieurs fois. Les scripts suivent l'état de la question et désactivent les commandes inappropriées après sa résolution ou son abandon.
- **Équilibre du mode défi :** les tours et les points devaient être suivis séparément pour chaque participant. Le jeu alterne les joueurs après chaque question et limite chaque joueur à cinq questions, sans compte à rebours.
- **Chargement des données :** les questions sont lues depuis un fichier JSON avec la fonction `fetch`. Pour éviter les restrictions du navigateur sur l'accès aux fichiers locaux, le site doit être lancé avec un serveur local tel que Live Server.
- **Affichage sur différents écrans et en arabe :** la mise en page doit rester lisible en mode droite-à-gauche, sur ordinateur comme sur mobile. Des règles CSS adaptatives, des zones de texte défilantes et des indications de focus clavier contribuent à préserver la lisibilité et l'accessibilité des commandes.

7. Conclusion

Ce projet nous a permis de consolider nos connaissances en HTML, CSS et JavaScript, notamment sur la structuration d'une page web, l'utilisation des sélecteurs CSS, la conception responsive et la gestion des interactions et de la logique du jeu avec JavaScript.
Il constitue une base solide pour aborder des projets de développement web plus avancés à l'avenir.

Références

[1] Fazaa, Tahar. *Énigmes de Tunisie*. Tunis : Sindbad éditions, 2014.

[2] MDN Web Docs. *CSS*. `https://developer.mozilla.org/en-US/docs/Web/CSS/`

[3] Stack Overflow. `https://stackoverflow.com`